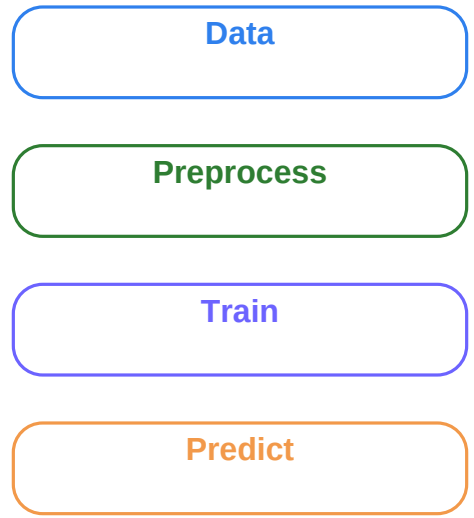
Reusable ML Pipeline

using scikit-learn + Random Forest

This guide explains what sklearn is, what a pipeline is, how train-test split works, how OneHotEncoder handles categorical data, and how RandomForestClassifier makes predictions.



Beginner note: The focus is reusability: train once, save the full workflow, and use it again for new data.

1. What is scikit-learn?

Think of sklearn as a ready-made ML toolbox for Python.

Preprocessing tools

- Fill missing values
- Scale numeric columns
- Encode text categories

Model training tools

- Logistic Regression
- Decision Trees
- Random Forest
- Many more algorithms

Evaluation tools

- Accuracy
- Precision / Recall
- F1-score
- Confusion matrix

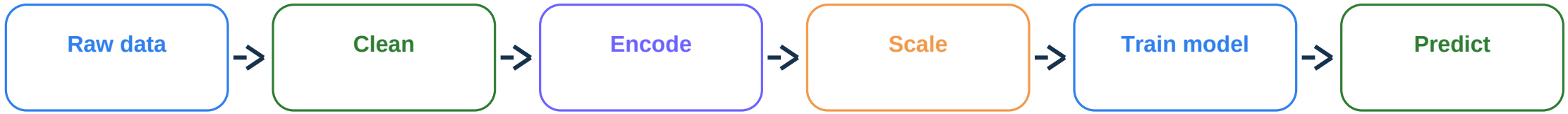
Common imports used in this project

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
```

Beginner note: sklearn helps us write reusable ML workflows instead of one-time scattered code.

2. What is a reusable ML pipeline?

A pipeline connects ML steps in a fixed order so the same workflow can run again.



Without pipeline

- Steps are separate
- Easy to forget a step
- Hard to reuse
- More manual mistakes

With pipeline

- Steps are connected
- Same order every time
- Easy to reuse
- Cleaner code

Production idea

- Save full workflow
- Load later
- Predict new data
- Keep logic consistent

Beginner note: A reusable pipeline should include preprocessing and model training together.

3. Train-test split in sklearn

We train on one part of the data and test on unseen data.

Code

```
from sklearn.model_selection import train_test_split

X = df.drop('Churn', axis=1)
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

X_train

Input features used to train the model.

X_test

Input features used only for testing.

y_train

Correct answers for the training rows.

y_test

Correct answers used to check predictions.

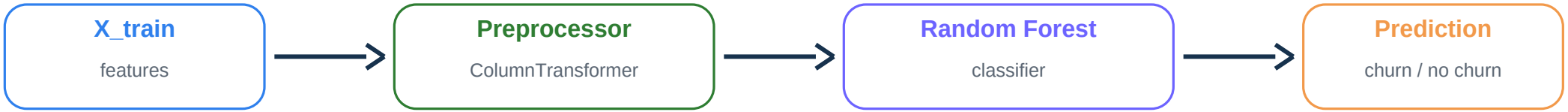
Why does test_size=0.2 mean 80:20?

0.2 means 20% of the full dataset is kept for testing. The remaining 80% is used for training. If you use test_size=0.3, it becomes 70% train and 30% test.

Beginner note: Always split before fitting the pipeline. Fit only on X_train and y_train.

4. Pipeline structure for churn prediction

The final pipeline should combine preprocessing and model training.



Main reusable pipeline

```
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(
        random_state=42
    ))
])

model_pipeline.fit(X_train, y_train)
```

What fit() does here

1. Learns missing-value rules from training data
2. Learns encoding categories from training data
3. Learns scaling rules from training data
4. Trains the Random Forest model

Beginner note: Do not clean, encode, and scale test data manually. Let the pipeline handle it.

5. Numerical preprocessing

Numeric columns usually need missing-value handling and scaling.

Age	Tenure	Monthly
25	12	850
NaN	7	650
42	NaN	1200

SimpleImputer(strategy="median")

Fills missing numeric values using the median. Example: [10, 12, missing, 20] becomes [10, 12, 12, 20].

StandardScaler()

Transforms numeric columns to a comparable scale. This is useful when columns have very different ranges, like age and monthly charges.

```
numeric_pipeline = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScal...
```

6. OneHotEncoder for categorical columns

Converts text categories into numeric 0/1 columns that ML models can understand.

Customer	ContractType
C1	Monthly
C2	Yearly
C3	Two-Year

Customer	Monthly	Yearly	Two-Year
C1	1	0	0
C2	0	1	0
C3	0	0	1

Why not 1, 2, 3?

Number labels create a fake order. Monthly, Yearly, and Two-Year are categories, not quantities.

What 0 and 1 mean

1 means yes for that category. 0 means no for that category.

handle_unknown="ignore"

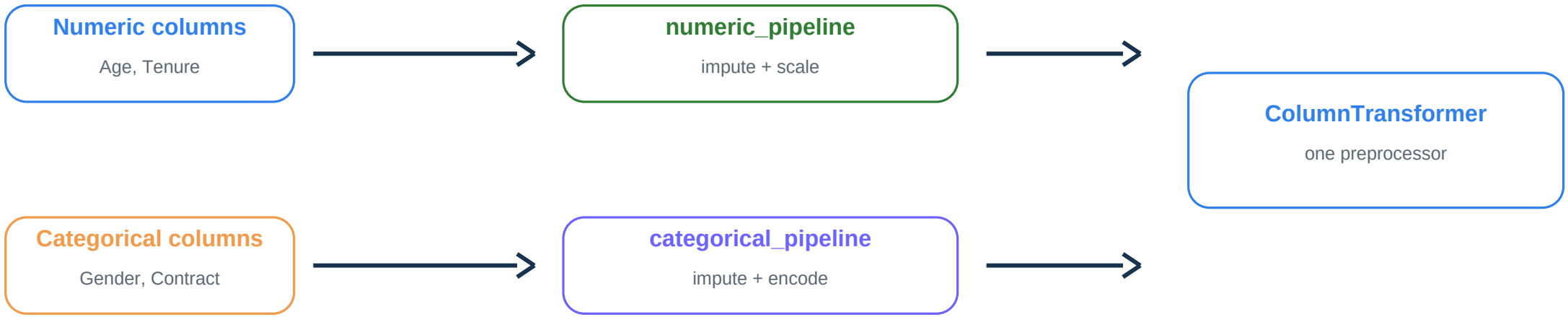
Prevents errors if future data contains a new category not seen during training.

Pipeline code

```
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

7. ColumnTransformer: combine both preprocessing paths

Real datasets have both numerical and categorical columns.



Code

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_pipeline, numeric_features),
    ('cat', categorical_pipeline, categorical_features)
])
```

Beginner note: ColumnTransformer keeps each column type in the correct preprocessing lane.

8. What is RandomForestClassifier?

A Random Forest is many decision trees working together.

Decision tree

One tree learns simple yes/no decision rules from the data.

Random forest

Many trees are trained slightly differently. Their votes are combined.

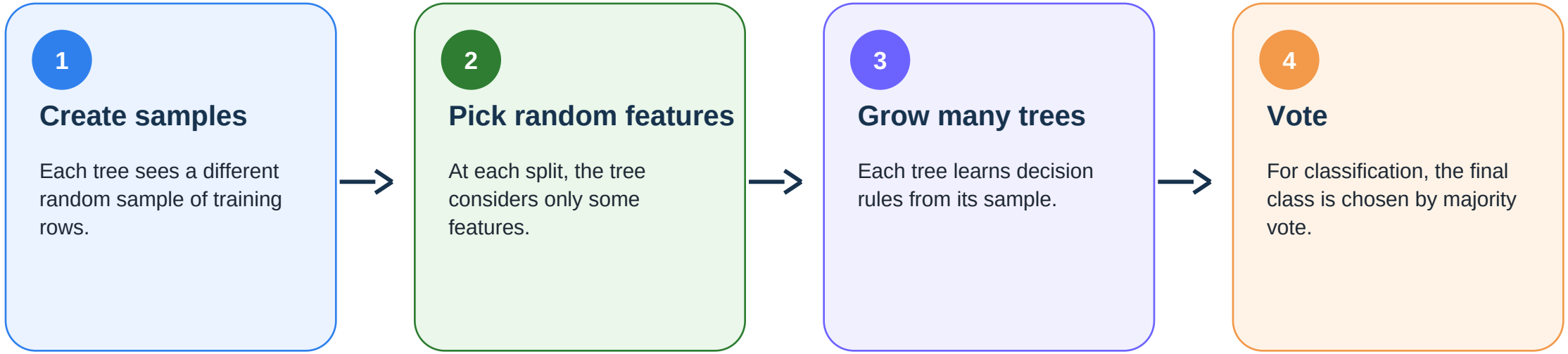
Classifier

For churn, it predicts classes such as Churn or No Churn.

Beginner note: Random Forest often performs well because it combines many weak decisions into a stronger prediction.

9. How Random Forest trains step by step

The model builds many trees and combines their decisions.



Basic code

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

Beginner note: n_estimators=100 means the forest uses 100 decision trees.

10. Complete reusable sklearn pipeline code

This is the core project code structure.

Part A - split + numeric preprocessing

```
X = df.drop('Churn', axis=1)
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

Part B - categorical + final pipeline

```
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipeline, numeric_features),
    ('cat', categorical_pipeline, categorical_features)
])

model_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(random_state=42))
])
```

Train, predict, evaluate

```
model_pipeline.fit(X_train, y_train)
y_pred = model_pipeline.predict(X_test)

print(classification_report(y_test, y_pred))
```

11. Evaluate and reuse the pipeline

The goal is not just training once; the pipeline should work again for future data.

Evaluation metrics

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

Why recall matters

In churn prediction, missing a likely churn customer can cause business loss. Recall helps measure how many churn customers were found.

Reusable output

Save the full pipeline, not only the model. The saved pipeline includes preprocessing plus model.

Save and load

```
import joblib

joblib.dump(model_pipeline, 'churn_pipeline.pkl')
loaded_pipeline = joblib.load('churn_pipeline.pkl')
```

Predict new customer data

```
new_prediction = loaded_pipeline.predict(new_custom...
print(new_prediction)
```

Beginner note: Because preprocessing is inside the pipeline, future data is handled consistently.

12. Mini Project 2 final checklist

Use this checklist before submission.

Requirement	Expected result
Train-test split	Uses train_test_split with test_size=0.2
Numeric preprocessing	SimpleImputer + StandardScaler inside pipeline
Categorical preprocessing	SimpleImputer + OneHotEncoder inside pipeline
ColumnTransformer	Separates numeric and categorical preprocessing
Model	RandomForestClassifier inside final Pipeline
Evaluation	Uses classification_report and confusion matrix
Reusability	Pipeline can predict new customer data
Documentation	Explains every major decision

Final takeaway

A reusable sklearn pipeline is a complete workflow: preprocessing + model training + prediction in one repeatable structure.